

Simple Movie Explorer & Watchlist Project Documentation

Purpose: Beginner-friendly React learning project that focusing on Redux Toolkit, API integration (OMDb), authentication, and infinite scrolling.

Project Overview

Project Name: Simple Movie Explorer & Watchlist

This is a frontend-only React application where users can:

- Log in using a authentication form (Redux-managed, no real backend).
- Search for movies via the OMDb API with **infinite scrolling** (loads more results as you scroll).
- Add movies from search results to a personal watchlist.
- View the watchlist, mark movies as **watched**, and **delete** them.
- Log out to reset the session.

Key Learning Focus:

Redux Toolkit (slices, thunks, state management), async API calls with Axios, infinite scroll, conditional rendering for auth, modular folder structure.

Tech Stack:

- React + Vite (fast development)
- Redux Toolkit + react-redux
- Axios (API requests)
- react-infinite-scroll-component
- TypeScript or plain JavaScript
- Basic CSS / Tailwind (optional)

Estimated Completion Time: 4–6 hours for beginners

Prerequisites: Basic React knowledge, Node.js installed

Installation & Dependencies

Get Free OMDb API Key

1. Go to: <http://www.omdbapi.com/apikey.aspx>
2. Choose **FREE** (1,000 requests/day limit)
3. Fill form → receive key via email (e.g. abcd1234)
4. Add to `.env` file (see below)

Project Setup

1. Create project

```
npm create vite@latest movie-explorer -- --template react-ts cd movie-explorer
```

2. Install dependencies

```
npm install @reduxjs/toolkit react-redux axios react-infinite-scroll-component
```

Optional: Tailwind

```
npm install -D tailwindcss postcss autoprefixer npx tailwindcss init -p
```

3. Create .env file in root:

```
VITE_OMDB_API_KEY=your_api_key_here
```

4. Start development server:

```
npm run dev
```

Folder Structure

```
movie-explorer/
├── public/
├── src/ | └── api/
│   ├── omdbApi.ts # Axios helpers for search
│   ├── components/
│   │   ├── Header.tsx
│   │   ├── LoginForm.tsx
│   │   ├── MovieCard.tsx
│   │   ├── MovieGrid.tsx # Infinite scroll container
│   │   ├── SearchBar.tsx
│   │   └── WatchlistItem.tsx
```

```

├── Watchlist.tsx
├── features/
│   ├── auth/
│   │   ├── authSlice.ts # login / logout
│   │   └── movies/
│   │       ├── moviesSlice.ts # search, watchlist, thunks
│   │       └── types.ts
│   └── store/
│       ├── index.ts # configureStore
│       └── pages/
│           ├── Discover.tsx # Search + infinite list
│           └── WatchlistPage.tsx
├── App.tsx
├── main.tsx # Provider wrapper
├── App.css / index.css
├── .env
├── package.json
└── README.md

```

Tasks to Perform (Step-by-Step for Interns)

1. Set up project + install dependencies + add .env with API key.
2. Create Redux store (store/index.ts) and wrap App in Provider (main.tsx).
3. Implement auth slice (authSlice.ts) + simple LoginForm component.
4. Create movies slice (moviesSlice.ts):
 - Async thunk to fetch movies from OMDb (fetchMovies)
 - Reducers: addToWatchlist, removeFromWatchlist, toggleWatched, setSearchTerm
5. Build API helper (omdbApi.ts) for search requests.
6. Create Discover page:
 - SearchBar → dispatch search term
 - MovieGrid with infinite scroll (using react-infinite-scroll-component)
 - MovieCard with "Add to Watchlist" button
7. Create Watchlist page:
 - Display list of added movies
 - Each item has toggle watched checkbox/button + delete button
8. In App.tsx:
 - If not authenticated → show LoginForm
 - If authenticated → show main content (Header + Discover + Watchlist sections/tabs)
9. Add basic styling (CSS or Tailwind) and test full flow: login → search movies → scroll to load more → add movie → go to watchlist → toggle watched → delete → logout
10. (Optional) Deploy to Netlify / Vercel and share live link.

Expected Final Output

After running the app (npm run dev):

Unauthenticated view:

- Clean login form (username input field + "Log In" button)

After successful login:

- Header: App title / Search bar / Watchlist link or button / Logout button
- Main area: Infinite-scroll movie grid (Discover section) → Search for e.g. "Avengers" → results load 10 at a time as you scroll → Each movie card shows: Poster (if available), Title, Year, "Add to Watchlist" button
- Ability to add movies to personal watchlist

Watchlist View:

- List of added movies (title, year, poster if available)
- Each item has:
 - "Watched" toggle (checkbox or button)
 - "Delete" button
- If empty: nice message like "Your watchlist is empty"

Behavior notes:

- API fetches real movie data from OMDb (e.g. search "Avengers" shows actual results)
- Errors handled gracefully:
 - "No results found" message for invalid search
 - Loading spinner/indicator while fetching
 - Rate-limit or API error message if something goes wrong

Key Learning Points

By finishing this project, you will understand:

1. Redux Toolkit basics: slices, reducers, actions, createAsyncThunk
2. Managing global state: separate slices for auth + movies/watchlist
3. Async data fetching: thunks, loading/error states, extraReducers
4. Infinite scrolling pattern: page tracking, hasMore flag, dispatching more data on scroll
5. Authentication flow: protected UI with conditional rendering
6. API key security: using .env variables (VITE_OMDB_API_KEY)
7. Modular React architecture: features / components / pages separation
8. Debugging: using Redux DevTools to inspect state changes live
9. Real-world API integration: handling pagination & API responses

